Applied Linear Algebra

Matrix Calculus for Learning Systems

Unit 3

Devi Prasad M
Manipal School of Information Sciences
Manipal

October 2026

Topics and Time Allocation

- Derivatives in one dimension (0.5 h)
- Gradients in multiple dimensions (1 h)
- Chain rule in multiple dimensions (1 h)
- Gradients of matrix expressions (2 h)
- Optimization using gradient descent (3.5 h)
  - single-sample updates
  - Logistic regression (No hidden layer)
  - Softmax regression (No hidden layer)
- Backpropagation (3 h)
  - Computational graphs
  - Multi-layer perceptrons (Hidden Layers)

Derivatives in One Dimension

A derivative is the rate of change of a function with respect to changes in its inputs.

For functions $f : \mathbb{R} \to \mathbb{R}$, the derivative of f at x is defined as:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Derivative - Rate of Change

$f'(x)$ measures how fast the output changes as the input changes:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

This limit tells us what the ratio between the perturbation h in the input and the change in the function value $f(x+h) - f(x)$ converges to as h shrinks to zero.

If the limit exists, we say that f is *differentiable* at x.

Derivatives in One Dimension

- **Local linearity.** Near any point, a smooth function behaves like a straight line:

$$f(x + \Delta x) \approx f(x) + f'(x)\,\Delta x$$

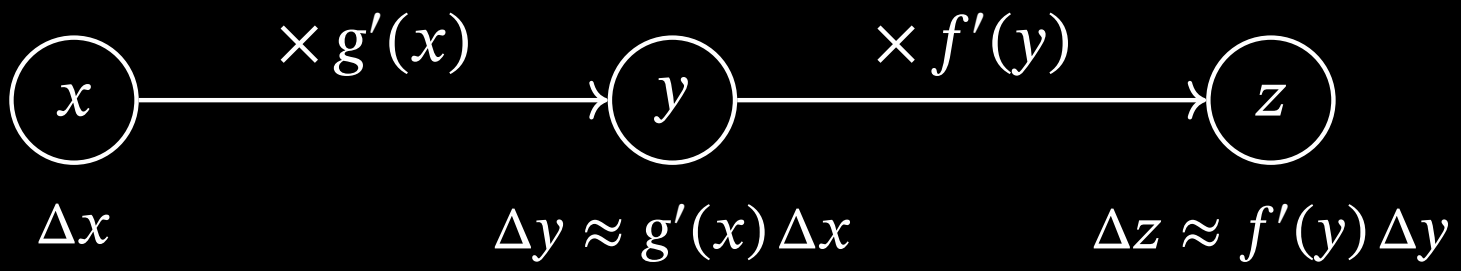
- **Direction of descent.** To decrease f, move x against the sign of $f'(x)$.
- At a minimum $f'(x) = 0$, though not every point with $f'(x) = 0$ is a minimum.

The Chain Rule

Rates of change multiply through a composition:

$$\frac{d}{dx} f(g(x)) = f'(g(x))\, g'(x)$$

Follow a small nudge. Write $y = g(x)$ and $z = f(y)$.



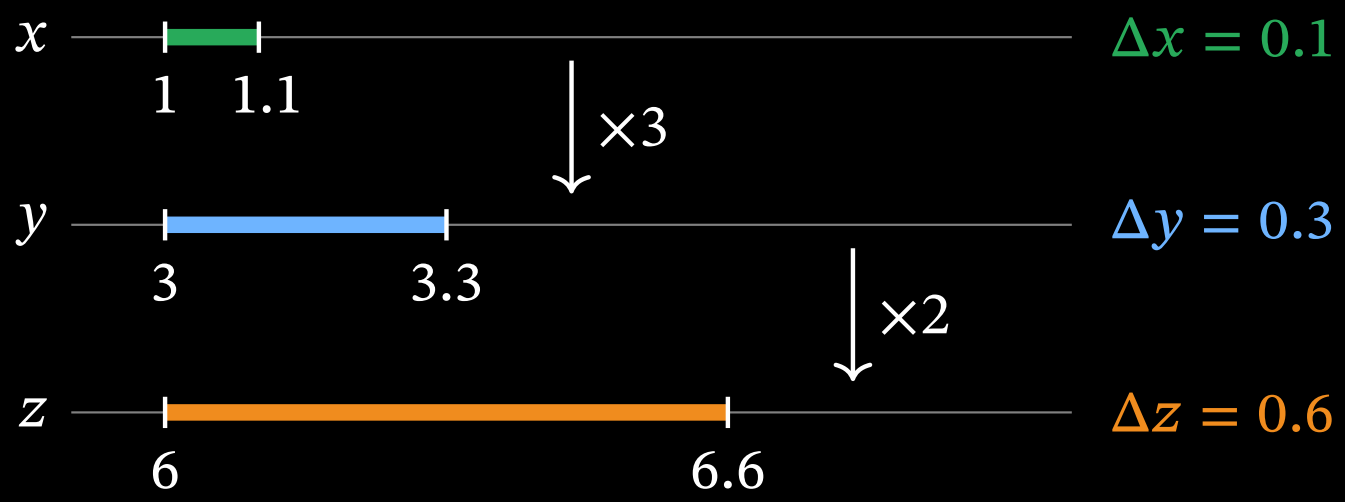
Substituting, $\Delta z \approx f'(y)\, g'(x)\, \Delta x$, so each stage scales the nudge and the scale factors multiply.

Example. If y changes $3\times$ as fast as x, and z changes $2\times$ as fast as y, then z changes $3 \times 2 = 6\times$ as fast as x.

Watch out. f' is evaluated at $y = g(x)$, where the nudge actually arrives, not at x.

The Chain Rule: Watching a Nudge Grow

Straight lines: $y = 3x$, $z = 2y$. Nudge x from 1 to 1.1.



The derivative is the **stretch factor**:

$$\frac{dy}{dx} = 3, \qquad \frac{dz}{dy} = 2, \qquad \frac{dz}{dx} = 3 \times 2 = 6.$$

Check: $z = 2(3x) = 6x$.

On Curves, the Chain Rule Holds Locally

$y = x^3$, $z = y^2$, starting at $x = 1$. The derivatives predict

$$\frac{dy}{dx} = 3x^2 = 3, \qquad \frac{dz}{dy} = 2y = 2, \qquad \frac{dz}{dx} = 3 \times 2 = 6.$$

Now nudge x by less and less:

Δx	Δy	$\Delta y/\Delta x$	Δz	$\Delta z/\Delta y$	$\Delta z/\Delta x$
0.1	0.331	3.31	0.7716	2.331	7.716
0.01	0.030301	3.0301	0.061520	2.0303	6.152
0.001	0.0030030	3.0030	0.0060150	2.0030	6.015
$\to 0$		$\to 3$		$\to 2$	$\to 6$

- **Why not exactly** 0.03**?** The slope predicts 0.03; the extra 0.000301 comes from the curve bending. Shrink the nudge 10× and the extra shrinks 100× (to 0.000003), so the slope's prediction takes over.
- In every row, $\dfrac{\Delta z}{\Delta x} = \dfrac{\Delta z}{\Delta y} \cdot \dfrac{\Delta y}{\Delta x}$ exactly. As $\Delta x \to 0$, the factors become 3 and 2.

Functions in One Dimension

Basic Functions

- Exponential: $\exp(z)$ — e^z
- Natural Log: $\ln(z)$

Activation Functions

- Sigmoid: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$
- Tanh: $\tanh(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}}$
- ReLU: $\max(0, z)$
- Leaky ReLU: $\max(0.01z, z)$
- GELU: $z \cdot \Phi(z)$

Derivative of e^z

$$\frac{d}{dz}(e^z) = \lim_{h \to 0} \frac{e^{z+h} - e^z}{h} \quad \text{(Limit definition)}$$

$$= \lim_{h \to 0} \frac{e^z \cdot e^h - e^z}{h} \quad \text{(Exponent rules)}$$

$$= \lim_{h \to 0} \frac{e^z(e^h - 1)}{h} \quad \text{(Factor out } e^z\text{)}$$

$$= e^z \cdot \lim_{h \to 0} \frac{e^h - 1}{h} \quad \text{(Move } e^z \text{ outside the limit)}$$

$$= e^z \cdot 1 \quad \text{(Using the identity } \lim_{h \to 0} \frac{e^h - 1}{h} = 1\text{)}$$

$$= e^z$$

Python snippet to verify $e^h \approx 1 + h$ for small h.

Derivative of the Natural Logarithm

$$\text{Let } y = \ln(z)$$

$$e^y = z \quad \text{(Rewrite in exponential form)}$$

$$\frac{d}{dz}(e^y) = \frac{d}{dz}(z) \quad \text{(Differentiate both sides)}$$

$$e^y \cdot \frac{dy}{dz} = 1 \quad \text{(Apply the chain rule)}^{\dagger}$$

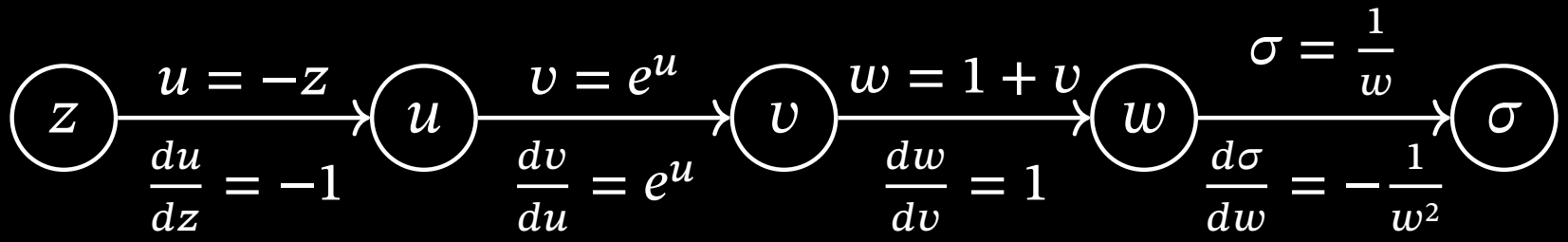
$$\frac{dy}{dz} = \frac{1}{e^y}$$

$$\frac{dy}{dz} = \frac{1}{z} \quad \text{(Substitute } e^y = z\text{)}$$

$^{\dagger} f(g(z)) = e^{g(z)},\ g(z) = y.$

Derivative of the Sigmoid via the Chain Rule

Simplify $\sigma(z) = 1/(1+e^{-z})$ into four steps. Operation is shown above the arrow, and the local derivative below the arrow.



Chain rule: multiply the local derivatives along the chain.

$$\frac{d\sigma}{dz} = \frac{d\sigma}{dw}\frac{dw}{dv}\frac{dv}{du}\frac{du}{dz} = \left(-\frac{1}{w^2}\right)(1)\,(e^u)\,(-1) = \frac{e^{-z}}{(1+e^{-z})^2}$$

Rewrite in terms of σ:

$$\frac{e^{-z}}{(1+e^{-z})^2} = \underbrace{\frac{1}{1+e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1+e^{-z}}}_{1-\sigma(z)} \quad \Longrightarrow \quad \boxed{\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)}$$

More About the Sigmoid

Show that $\sigma(-z) = 1 - \sigma(z)$

$$\textbf{Note } \sigma(z) = \frac{1}{1+e^{-z}} \quad \textbf{and} \quad \sigma(-z) = \frac{1}{1+e^{z}}$$

$$1 - \sigma(z) = 1 - \frac{1}{1+e^{-z}}$$

$$= \frac{e^{-z}}{1+e^{-z}} = \frac{1}{e^{z}} \cdot \frac{1}{1+e^{-z}}$$

$$= \frac{1}{1+e^{z}}$$

$$= \sigma(-z)$$

Derivative of $\sigma(-z)$

$$\begin{aligned}\frac{d}{dz}\sigma(-z) &= \frac{d}{dz}\big(1-\sigma(z)\big)\\ &= 0-\sigma'(z)\\ &= -\sigma(z)\big(1-\sigma(z)\big)\end{aligned}$$

- $\sigma'(-z)$: take σ', then evaluate it at $-z$.

$$\sigma'(-z) = \sigma(-z)\big(1-\sigma(-z)\big) \;>\; 0$$

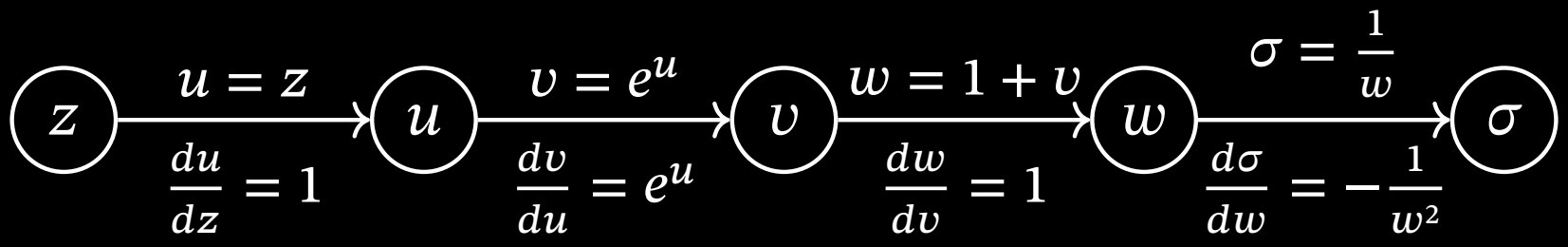
- $\frac{d}{dz}\,\sigma(-z)$: differentiate the composite with respect to z.

$$\frac{d}{dz}\,\sigma(-z) = -\,\sigma'(-z) \;<\; 0$$

The two differ by exactly the chain-rule factor $\frac{d}{dz}(-z) = -1$.

Derivative of $\sigma(-z)$ Using the Chain Rule

Simplify $\sigma(-z) = 1/(1+e^z)$ into four steps.



Chain rule: multiply the local derivatives along the chain.

$$\frac{d\sigma}{dz} = \frac{d\sigma}{dw}\frac{dw}{dv}\frac{dv}{du}\frac{du}{dz} = \left(-\frac{1}{w^2}\right)(1)\,(e^u)\,(1) = -\frac{e^z}{(1+e^z)^2}$$

Rewrite in terms of σ:

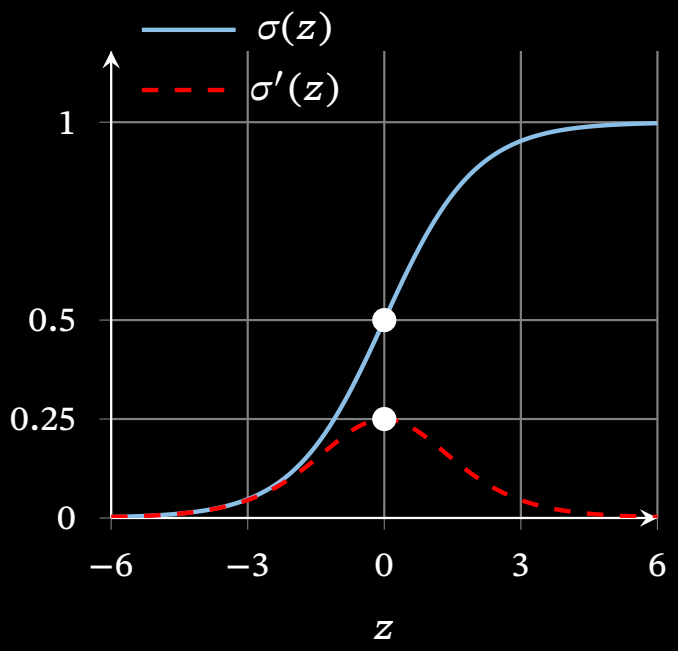
$$-\frac{e^z}{(1+e^z)^2} = -\underbrace{\frac{1}{1+e^z}}_{\sigma(-z)} \cdot \underbrace{\frac{e^z}{1+e^z}}_{1-\sigma(-z)} \quad \Longrightarrow \quad \boxed{\sigma'(-z) = -\sigma(-z)\big(1-\sigma(-z)\big)}$$

Derivative of $\sigma(-z)$ — One More Time!

Show the two forms of $\frac{d}{dz}\sigma(-z)$ are equivalent:

$$\begin{aligned}
\frac{d}{dz}\sigma(-z) &= -\sigma(z)\big(1-\sigma(z)\big) \\
&= -\big(1-\sigma(-z)\big)\,\sigma(-z) \quad \text{(Use } \sigma(-z) = 1-\sigma(z)\text{)} \\
&= -\sigma(-z)\big(1-\sigma(-z)\big) \quad \text{(Rearrange)}
\end{aligned}$$

The Sigmoid and Its Derivative



- **Range.** $0 < \sigma(z) < 1$ and $\sigma(0) = \frac{1}{2}$, so the output reads as a probability.
- **Increasing.** $\sigma'(z) > 0$ for every z.
- **Bounded slope.** $\sigma'(z) \le \frac{1}{4}$, largest at $z = 0$: $p(1-p)$ peaks at $p = \frac{1}{2}$.
- **Symmetry.** $\sigma(-z) = 1 - \sigma(z)$, so $\sigma'(-z) = \sigma'(z)$.
- **Flat tails.** $\sigma'(z) \to 0$ as $|z| \to \infty$.

Saturation: When the Gradient Vanishes

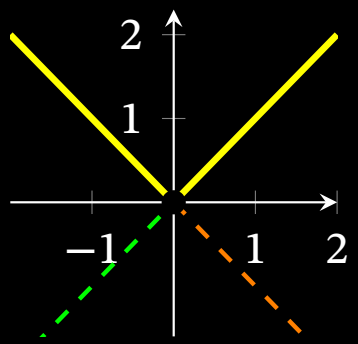
z	$\sigma(z)$	$\sigma'(z)$
0	0.5	0.25
2	0.881	0.105
4	0.982	0.018
6	0.998	0.0025
10	0.99995	0.000045

- **Saturation.** For large $|z|$ the output barely moves, so any gradient that passes through σ' is tiny and learning slows.
- **Deep networks.** Each sigmoid layer scales the backward signal by $\sigma' \le \frac{1}{4}$, so it can shrink layer after layer.

The loss decides. For $p = \sigma(\mathbf{w}^\top \mathbf{x})$:

- Cross-entropy: $\nabla_{\mathbf{w}} \ell = (p - y)\,\mathbf{x}$. The σ' factor cancels.
- Squared error: $\nabla_{\mathbf{w}} \ell = (p - y)\,p(1 - p)\,\mathbf{x}$. Nearly zero when the model is confidently wrong.

When Left and Right Slopes Disagree



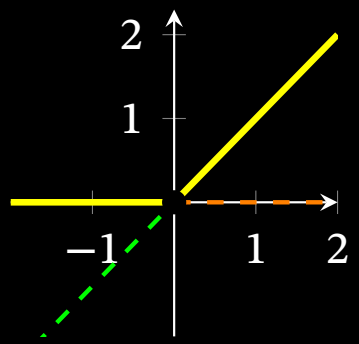
Absolute value

$f(x) = |x|$

kink at $x = 0$

$f'_-(0) = -1$

$f'_+(0) = +1$



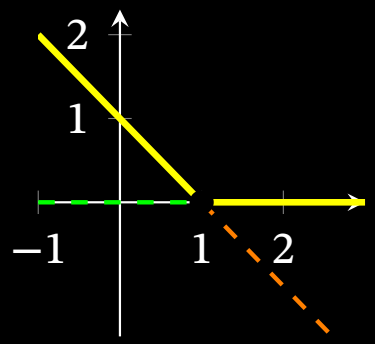
ReLU

$f(x) = \max(0, x)$

kink at $x = 0$

$f'_-(0) = 0$

$f'_+(0) = 1$



Hinge loss

$f(x) = \max(0, 1-x)$

kink at $x = 1$

$f'_-(1) = -1$

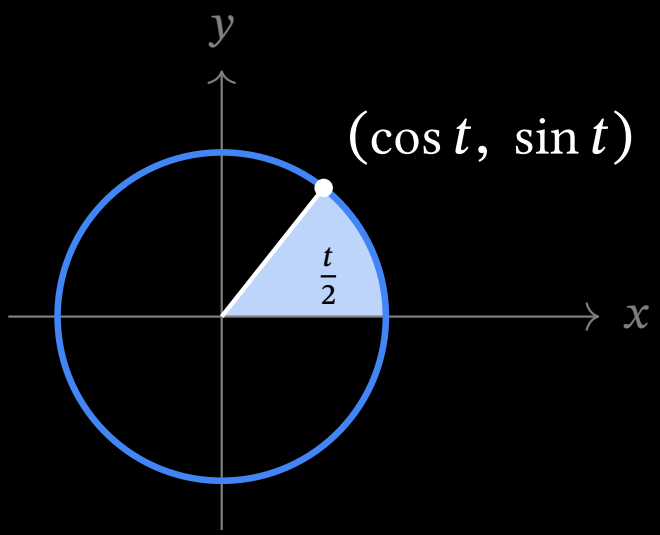
$f'_+(1) = 0$

Dashed lines continue each side's slope past the kink: left slope and right slope.

Two different lines: no single tangent, f' does not exist there.

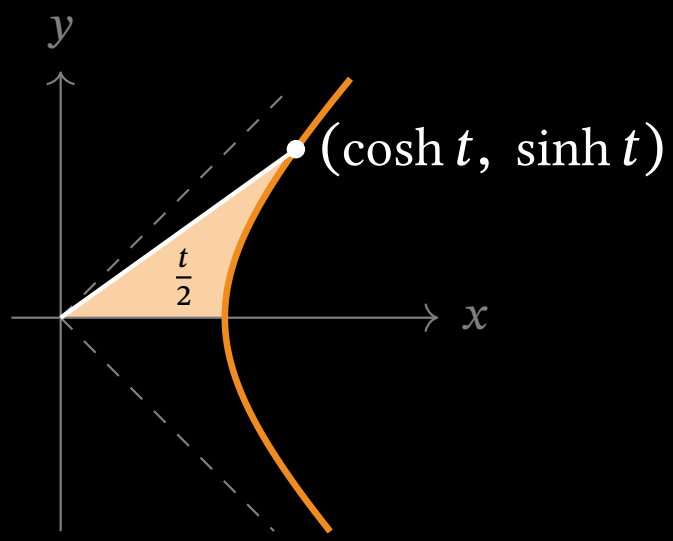
From Circle to Hyperbola

Circle $x^2 + y^2 = 1$



$$\cos^2 t + \sin^2 t = 1$$

Hyperbola $x^2 - y^2 = 1$



$$\cosh^2 t - \sinh^2 t = 1$$

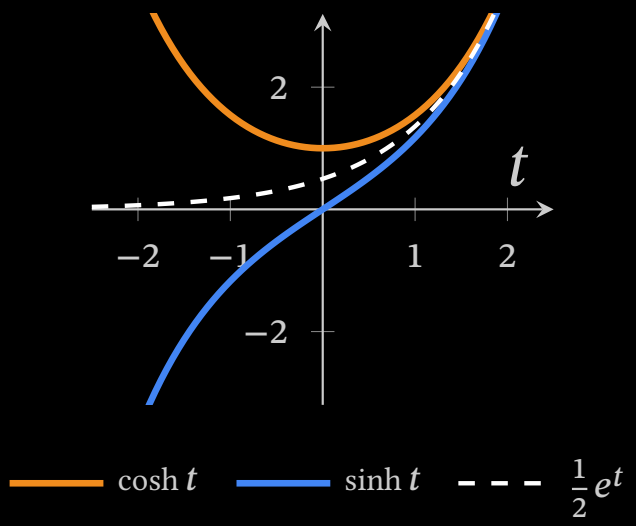
Shaded area $= \frac{t}{2}$ in both pictures. The hyperbolic pair is built from exponentials:

$$\cosh t = \frac{e^t + e^{-t}}{2}, \qquad \sinh t = \frac{e^t - e^{-t}}{2}.$$

Derivatives Without Sign Flips

Differentiate the definitions directly:

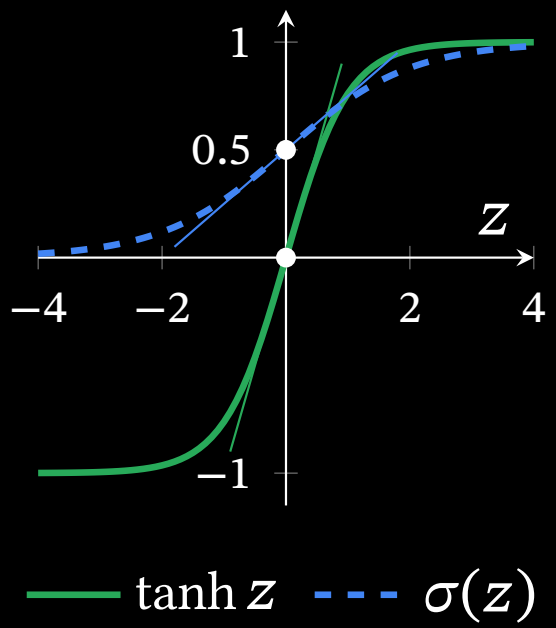
$$\frac{d}{dt}\cosh t = \frac{e^t - e^{-t}}{2} = \sinh t, \qquad \frac{d}{dt}\sinh t = \frac{e^t + e^{-t}}{2} = \cosh t.$$



- Trigonometry has a sign flip, $(\cos t)' = -\sin t$. The hyperbolic pair has **none**.
- $e^t = \cosh t + \sinh t$: cosh is the **even** part of e^t, sinh the **odd** part.
- For large t, both approach $\frac{1}{2}e^t$ (dashed).

tanh: A "Better" Sigmoid

$$\tanh z = \frac{\sinh z}{\cosh z} = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



- **Same S-shape:** $\tanh z = 2\,\sigma(2z) - 1$, the sigmoid stretched to $(-1, 1)$.
- **Zero-centred:** $\tanh 0 = 0$, outputs of both signs. σ outputs are always positive.
- **Steeper:** slope at 0 is 1, versus $\frac{1}{4}$ for σ (thin lines).
- **Still saturates:** flat for large $|z|$, so gradients still vanish there.

Check: $2\sigma(2z) - 1 = \dfrac{1 - e^{-2z}}{1 + e^{-2z}}$; multiply top and bottom by e^z to get $\tanh z$.

Recap: The Quotient Rule

For $h(x) = \dfrac{f(x)}{g(x)}$, wherever $g(x) \neq 0$:

$$h'(x) = \frac{f'(x)\,g(x) - f(x)\,g'(x)}{g(x)^2}$$

- The order matters: the term with f' comes first.
- The denominator is the original denominator, squared.

Example. $h(x) = \dfrac{x}{1+x^2}$, with $f = x$, $g = 1 + x^2$:

$$h'(x) = \frac{1 \cdot (1+x^2) - x \cdot 2x}{(1+x^2)^2} = \frac{1 - x^2}{(1+x^2)^2}$$

Derivative of Hyperbolic Tangent

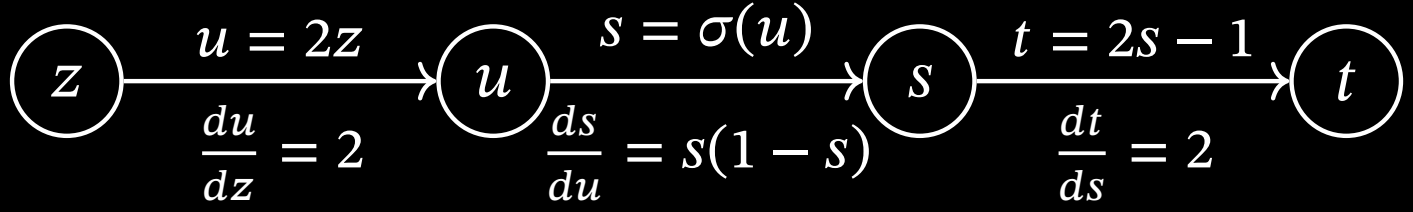
$$\frac{d}{dz}\tanh z = \frac{d}{dz}\frac{\sinh z}{\cosh z}$$

$$= \frac{\cosh z \cdot \cosh z - \sinh z \cdot \sinh z}{\cosh^2 z} \qquad \text{(quotient rule)}$$

$$= \frac{\cosh^2 z}{\cosh^2 z} - \frac{\sinh^2 z}{\cosh^2 z} = 1 - \tanh^2 z$$

Derivative of tanh using the Chain Rule

Start from $\tanh z = 2\,\sigma(2z) - 1$ and break it into three steps. Above each arrow is the operation; below it, the local derivative.



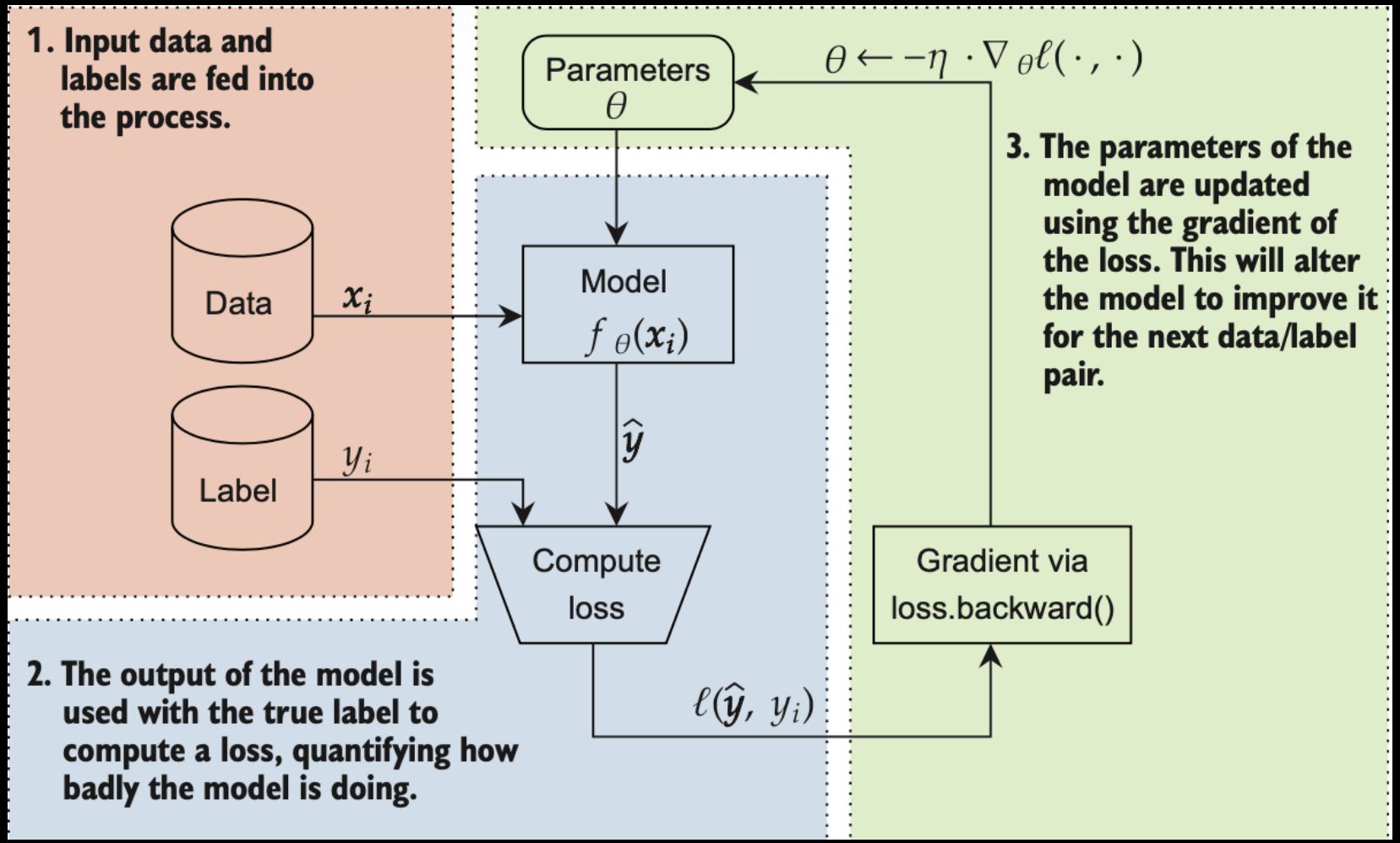
Chain rule: multiply the local derivatives.

$$\frac{d}{dz}\tanh z = \frac{dt}{ds}\,\frac{ds}{du}\,\frac{du}{dz} = 2 \cdot s(1-s) \cdot 2 = 4\,\sigma(2z)\big(1-\sigma(2z)\big)$$

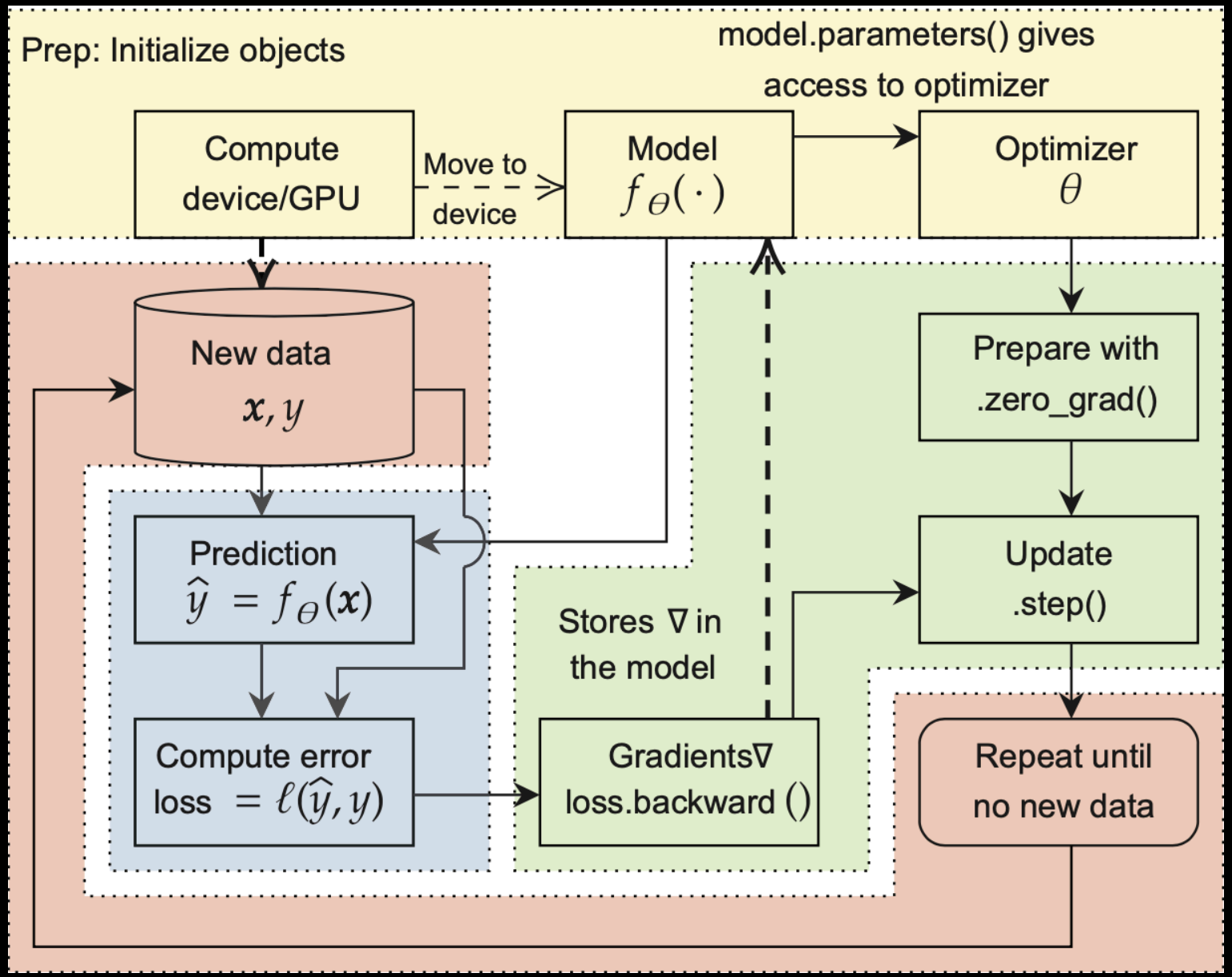
Rewrite in terms of tanh: since $t = 2s - 1$, $s = \frac{1+\tanh z}{2}$ and $1 - s = \frac{1-\tanh z}{2}$.

$$4 \cdot \frac{1+\tanh z}{2} \cdot \frac{1-\tanh z}{2} = (1+\tanh z)(1-\tanh z) = \boxed{1 - \tanh^2 z}$$

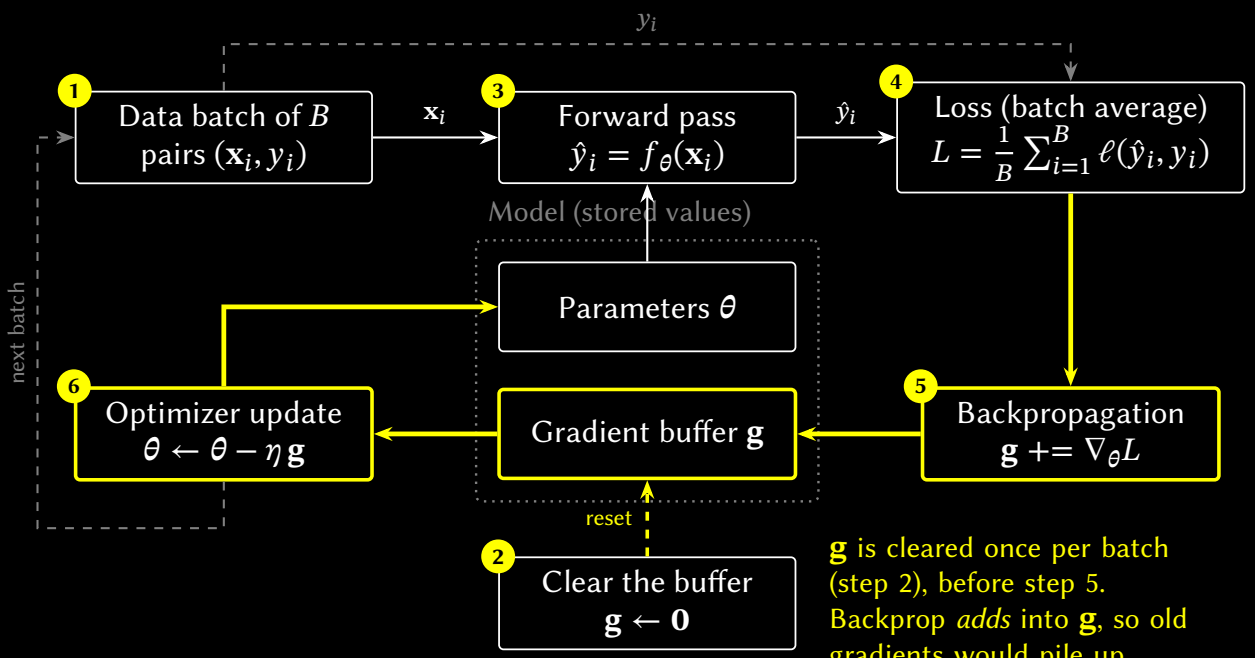
Training a Network Using PyTorch



Training a Network Using PyTorch

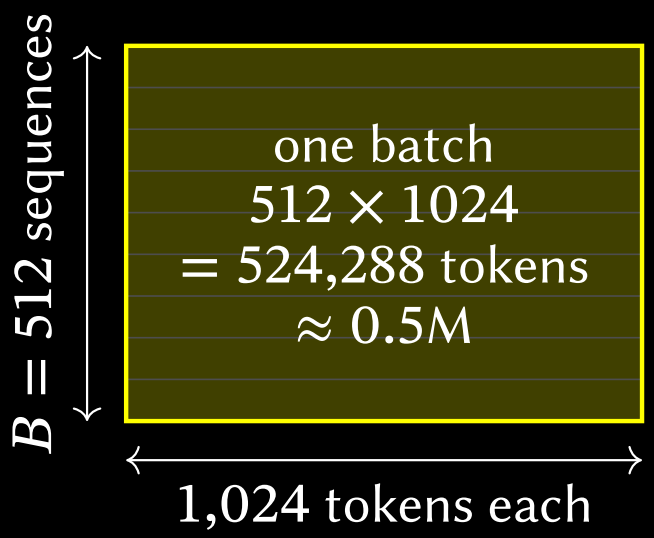


Where gradients are computed and used



In an LLM: a batch is B token sequences; steps 3–5 run once per batch, not once per sample, and give one gradient entry per parameter, billions in all.

GPT-2: one batch in numbers



Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

- "a larger batchsize of 512 is used"; context raised from 512 to 1,024 tokens.
- Vocabulary 50,257; WebText: about 8M documents, 40 GB.
- Learning rate "manually tuned" for each model.

Radford et al. (2019), Sec. 2.3 and Table 2. Total training tokens are not stated in the paper.

In an LLM: one forward pass, one backward pass and one update $\theta \leftarrow \theta - \eta g$ per batch of about 0.5M tokens.

GPT-3: the batch grows with the model

Parameters	Layers	d_{model}	Batch (tokens)	Peak LR	Updates ≈
125M	12	768	0.5M	6.0×10^{-4}	600,000
350M	24	1024	0.5M	3.0×10^{-4}	600,000
760M	24	1536	0.5M	2.5×10^{-4}	600,000
1.3B	24	2048	1M	2.0×10^{-4}	300,000
2.7B	32	2560	1M	1.6×10^{-4}	300,000
6.7B	32	4096	2M	1.2×10^{-4}	150,000
13B	40	5140	2M	1.0×10^{-4}	150,000
175B	96	12288	3.2M	0.6×10^{-4}	94,000

Context 2,048 tokens; 300B training tokens for every model.
Updates ≈ tokens / batch.
Larger model ⇒ larger batch, *smaller* learning rate: the paper notes this trend.

Brown et al. (2020), Table 2.1. The "Updates" column is derived, not from the paper.

In an LLM: at 175B, a batch of 3.2M tokens is about 1,560

Llama 2 and Llama 3: millions of tokens per update

Unit 3

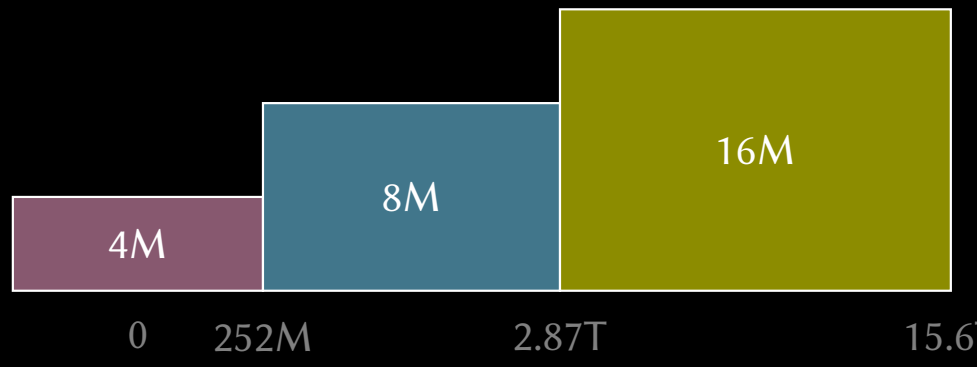
Llama 2 (7B to 70B)

- Global batch 4M tokens, context 4,096, 2T tokens in all: about 500,000 updates.
- AdamW, cosine schedule, 2,000 warmup steps, gradient clipping 1.0.
- Peak LR 3×10^{-4} (7B, 13B) and 1.5×10^{-4} (34B, 70B).

Llama 3

405B: the batch is ramped up

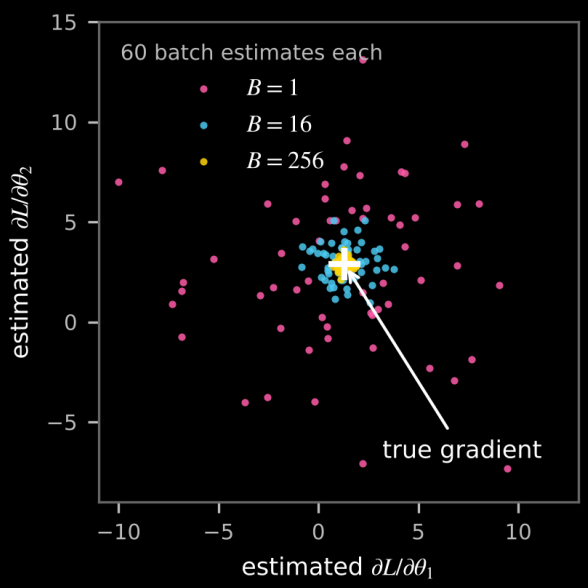
batch (tokens), sequences 4,096 → 8,192; not to sc



- AdamW, peak LR 8×10^{-5}, 8,000 warmup steps, cosine decay to 8×10^{-7}.

Touvron et al. (2023), Sec. 2.2; Llama Team (2024), Sec. 3.4.1. The Llama 3 paper says "8M sequences of 8,192 tokens"; read here as 8M tokens.

Why the batch size is a hyperparameter



- **Gradient quality.** $\mathbf{g}$ is an average over B samples. The scatter shrinks roughly like $1/\sqrt{B}$ (synthetic example, same true gradient).
- **Learning rate.** The best η depends on B; GPT-3 pairs larger batches with smaller learning rates.
- **Cost.** Memory and hardware grow with B.
- **Number of updates.** With a fixed token budget, updates $\approx$ tokens / B.
- **Does not fit?** Split the batch into micro-batches and keep $\mathbf{g}$ (do not clear it) until the last one: gradient accumulation.

Learning is optimization in high dimensions

A deep network has N adjustable parameters θ (billions in an LLM). A loss $L(\theta)$ scores it on training data; training finds θ with small L by going downhill in $\mathbb{R}^N$.

$L(\theta)$ (that is f in the pictures) is one number computed from all N parameters

The picture: vary θ_i, θ_j (x, y), freeze the other $N-2$



→ a surface $f(x, y)$ we can draw

Partial derivative: vary only θ_i (x), freeze the other $N-1$



→ one curve, one slope $\partial L/\partial\theta_i$

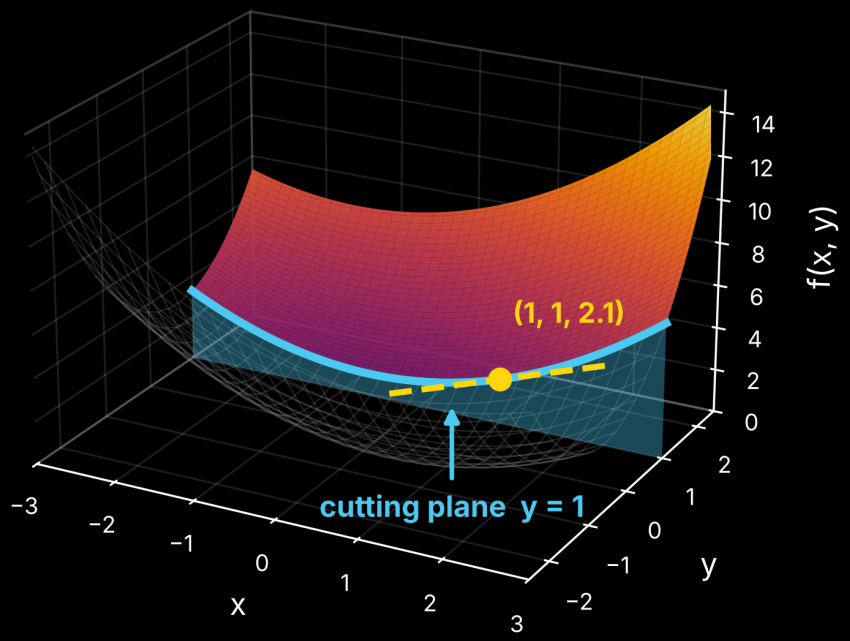
Gradient and training step: all N parameters at once



→ ∇L by backpropagation (next unit): N entries; then update $\theta \leftarrow \theta - \eta\nabla L$

Pictures vs. reality: our pictures show two parameters at a time; every definition that follows works for all N.

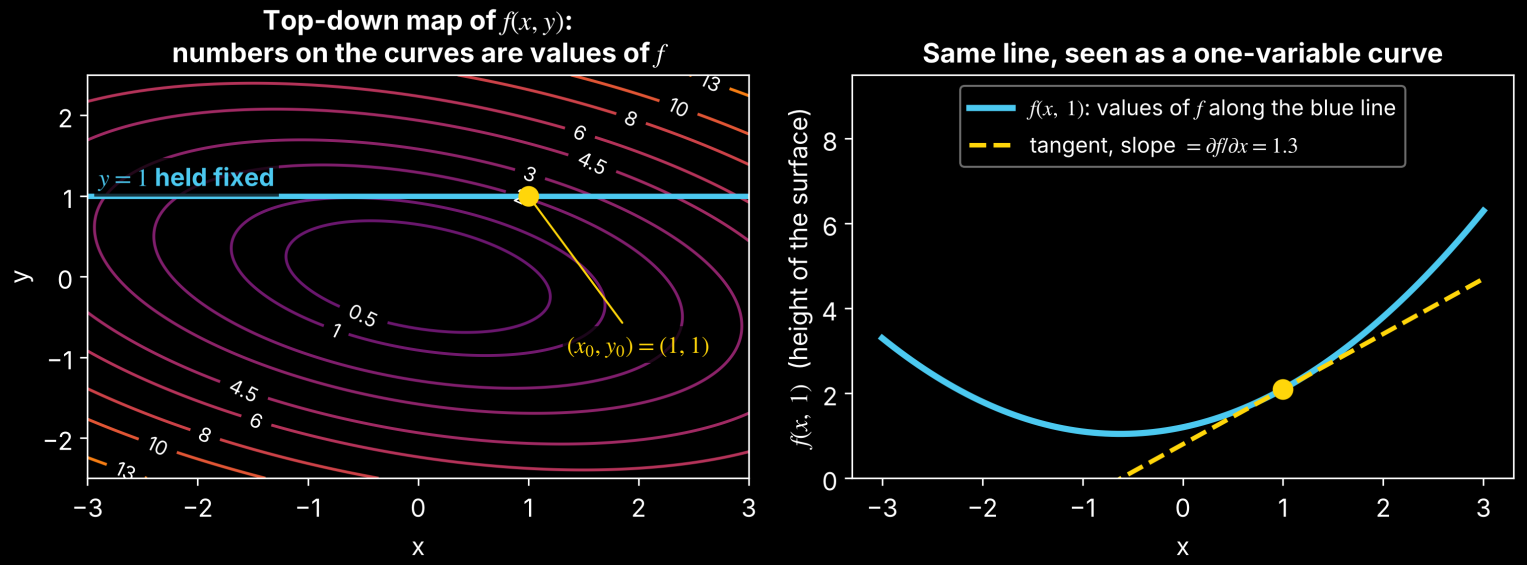
Slicing the loss: freeze all but one parameter



- A surface needs two inputs, so the picture varies x and y and freezes the rest.
- Cut it with the plane $y = 1$: only x varies, and what is left is an ordinary curve.
- Its slope at the yellow point is the partial derivative $\partial f / \partial x$.

In an LLM: pick one weight out of billions, freeze all the others and vary it: the loss traces a curve.

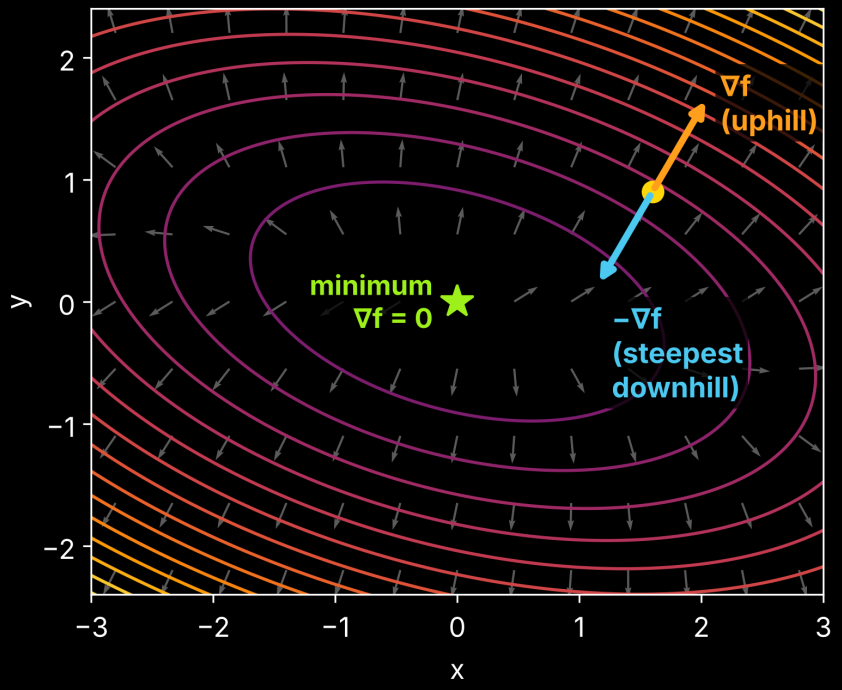
The partial derivative



$$\frac{\partial f}{\partial x_j}(\mathbf{x}) = \lim_{h \to 0} \frac{f(\mathbf{x} + h\,\mathbf{e}_j) - f(\mathbf{x})}{h}$$: change input j by a little and hold all the others fixed.

In an LLM: $\partial L / \partial \theta_j$ is the sensitivity of the loss to one weight.

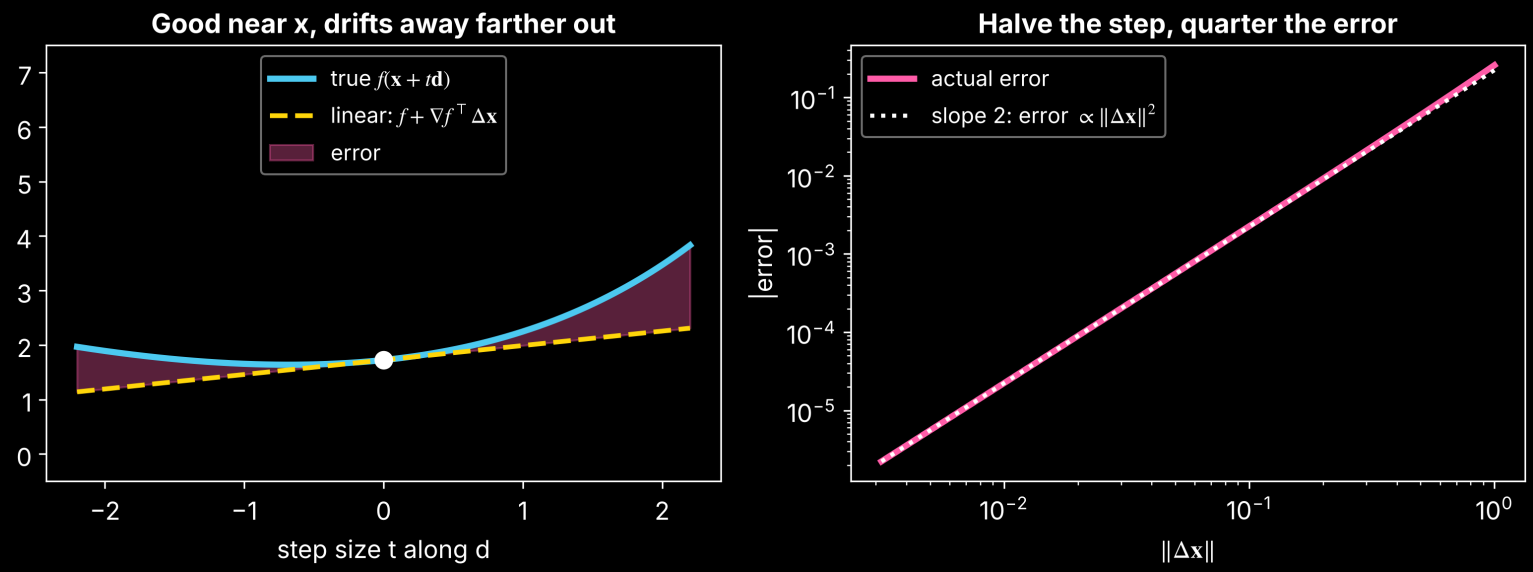
The gradient



- Collect the partials into one vector with the same shape as the input: $\nabla f(\mathbf{x}) = [\partial f/\partial x_1, \ldots, \partial f/\partial x_n]^\mathsf{T}$.
- ∇f points in the direction of fastest increase and is perpendicular to the contour lines.
- So $-\nabla f$ is the steepest way downhill.

In an LLM: ∇L has one entry per parameter, billions in all; $-\nabla L$ is the direction of steepest decrease.

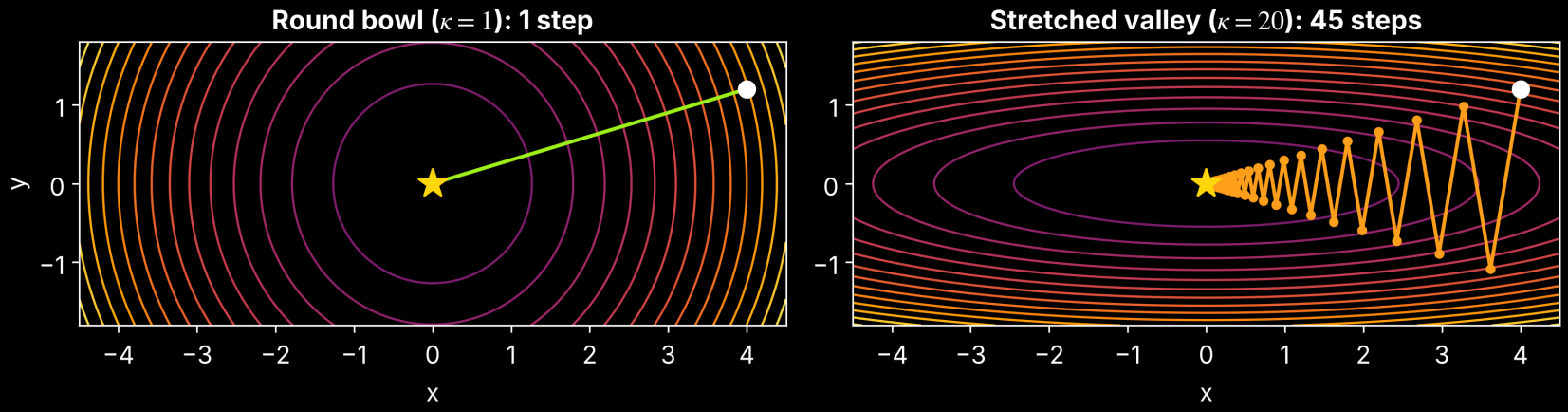
Local linearity and the downhill step



$f(\mathbf{x}+\Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^{\intercal}\Delta\mathbf{x}$. Take $\Delta\mathbf{x} = -\eta\,\nabla f$: then $\Delta f \approx -\eta\,\|\nabla f\|^2$, a decrease for small η.

In an LLM: $\theta \leftarrow \theta - \eta\nabla L$ lowers L when the step is small enough; in practice ∇L is estimated on a batch of text.

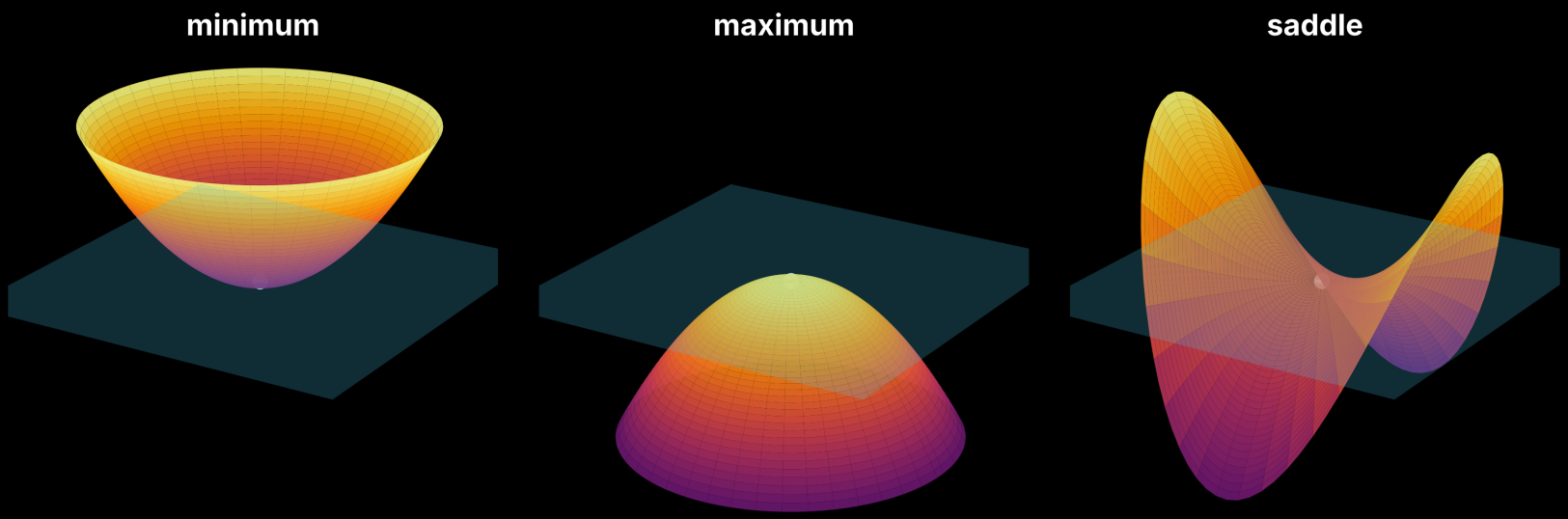
Going downhill: conditioning matters



$f = \frac{1}{2}(x^2 + \kappa y^2)$, with $\kappa = \lambda_{\max}/\lambda_{\min}$, the ratio of steepest to shallowest curvature. Each run uses its best fixed step size, $2/(\lambda_{\max} + \lambda_{\min})$, and stops within 0.05 of the minimum. The round bowl needs one step; the stretched valley zigzags and takes 45.

In an LLM: loss landscapes can be very stretched in some directions; this is one reason training uses optimizers beyond plain gradient descent, such as Adam (later).

A zero gradient does not certify a minimum



At any minimum, maximum or saddle, $\nabla f = \mathbf{0}$; the Hessian (coming later) tells them apart. The cyan plane is the flat tangent plane at the critical point.

In an LLM: training stops when the compute budget runs out, not at an exact minimum; in high dimensions, saddles are plausibly far more common than local minima.

Gradients in Multiple Dimensions

- **Partial derivative.** $\partial f / \partial x_j$: change one input, hold all the others fixed.
- **Gradient.** Collect the partials into a vector with the same shape as the input: $\nabla f(\mathbf{x}) = \left[\partial f / \partial x_1, \dots, \partial f / \partial x_n\right]^\top$.
- **Local linearity.** $f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \Delta\mathbf{x}$.
- **Steepest ascent.** ∇f points in the direction of fastest increase and is perpendicular to the contour lines, so $-\nabla f$ is the steepest way downhill. At a minimum, $\nabla f = \mathbf{0}$.
- **Jacobian.** For $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the $m \times n$ matrix with entries $J_{ij} = \partial f_i / \partial x_j$ records how every output responds to every input.

Chain Rule in Multiple Dimensions

- **Change flows through intermediate variables.** If $\mathbf{x} \to \mathbf{y} = \mathbf{g}(\mathbf{x}) \to z = f(\mathbf{y})$, a change in x_j reaches z through every y_i that depends on x_j.
- **Sum over paths.** $\dfrac{\partial z}{\partial x_j} = \displaystyle\sum_i \frac{\partial z}{\partial y_i}\,\frac{\partial y_i}{\partial x_j}$: multiply along each path, then add across paths.
- **Matrix form.** Jacobians multiply: $\mathbf{J}_{\mathbf{f}\circ\mathbf{g}}(\mathbf{x}) = \mathbf{J}_{\mathbf{f}}\big(\mathbf{g}(\mathbf{x})\big)\,\mathbf{J}_{\mathbf{g}}(\mathbf{x})$.
- **Scalar output.** When the final output is one number (a loss), $\nabla_{\mathbf{x}} z = \mathbf{J}_{\mathbf{g}}(\mathbf{x})^{\top} \nabla_{\mathbf{y}} z$: the gradient is passed backward through the transposed Jacobian.

Gradients of Matrix Expressions

- **Shape rule.** The gradient of a scalar with respect to a vector or matrix has the same shape as that vector or matrix. Checking shapes catches most errors.
- **Method.** Write the expression with indices, differentiate a single entry, then reassemble the result in matrix form.
- **Standard results.** $\nabla_{\mathbf{x}}(\mathbf{a}^{\mathsf{T}}\mathbf{x}) = \mathbf{a}$, $\nabla_{\mathbf{x}}(\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^{\mathsf{T}})\mathbf{x}$, $\nabla_{\mathbf{x}}\|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2 = 2\mathbf{A}^{\mathsf{T}}(\mathbf{A}\mathbf{x} - \mathbf{b})$.
- **Linear layer.** For $\mathbf{z} = \mathbf{W}\mathbf{x}$ and a scalar loss L: $\nabla_{\mathbf{x}} L = \mathbf{W}^{\mathsf{T}} \nabla_{\mathbf{z}} L$ and $\nabla_{\mathbf{W}} L = (\nabla_{\mathbf{z}} L)\, \mathbf{x}^{\mathsf{T}}$, an outer product.
- **Elementwise functions.** For $\mathbf{h} = \phi(\mathbf{z})$ applied entry by entry, the Jacobian is diagonal, so $\nabla_{\mathbf{z}} L = \nabla_{\mathbf{h}} L \odot \phi'(\mathbf{z})$.

Optimization Using Gradient Descent

- **Learning as minimization.** Training means choosing parameters θ that minimize the average loss $L(\theta) = \frac{1}{N} \sum_{n=1}^{N} \ell_n(\theta)$.
- **Update rule.** $\theta \leftarrow \theta - \eta \nabla L(\theta)$. By local linearity, a small step along $-\nabla L$ lowers the loss. If η is too small, progress is slow; if too large, the steps overshoot or diverge.
- **Single-sample updates.** $\theta \leftarrow \theta - \eta \nabla \ell_n(\theta)$ for a randomly chosen example n. Each step is cheap, and the gradient is noisy but correct on average.

Optimization Using Gradient Descent

- **Logistic regression.** $p = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ with cross-entropy loss gives $\nabla_{\mathbf{w}} \ell = (p - y)\,\mathbf{x}$: prediction error times input.
- **Softmax regression.** $\mathbf{p} = \mathrm{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$ with cross-entropy loss gives $\nabla_{\mathbf{W}} \ell = (\mathbf{p} - \mathbf{y})\,\mathbf{x}^\top$ for one-hot $\mathbf{y}$, the same pattern extended to K classes. Both losses are convex, so every local minimum is a global one.

Backpropagation

- **Computational graph.** The model and loss are written as a chain of simple operations. Each node needs only its own local derivative.
- **Forward, then backward.** The forward pass computes and stores every intermediate value. The backward pass applies the chain rule from the loss toward the inputs, reusing those stored values.
- **Error signals.** Each node receives $\partial L/\partial(\text{node})$ from the nodes it feeds. When a value feeds several nodes, the incoming signals add.
- **Efficiency.** One backward pass gives the gradient for every parameter, at a cost comparable to one forward pass.

Backpropagation

- **Multi-layer perceptron.** With $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$, $\mathbf{h} = \phi(\mathbf{z}_1)$, $\mathbf{p} = \mathrm{softmax}(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$:

$$\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{y},$$

$$\nabla_{\mathbf{W}_2} L = \boldsymbol{\delta}_2 \mathbf{h}^\top,$$

$$\boldsymbol{\delta}_1 = (\mathbf{W}_2^\top \boldsymbol{\delta}_2) \odot \phi'(\mathbf{z}_1),$$

$$\nabla_{\mathbf{W}_1} L = \boldsymbol{\delta}_1 \mathbf{x}^\top$$

- **Non-convexity.** Hidden layers make the loss non-convex. Gradient descent reaches a local solution, and that solution depends on the initialization.

Reference Books

- Michael Nielsen, *Neural Networks and Deep Learning*
  http://neuralnetworksanddeeplearning.com/
- François Chollet, *Deep Learning with Python*,
  https://www.manning.com/books/deep-learning-with-python
- 3Blue1Brown
  https://www.3blue1brown.com/?topic=calculus